# Neural Control Variates for Arithmetic Asian Option Pricing: Statistical Efficiency, Computational Cost and Reuse

Jed Dahlke

MSc Mathematical Finance Dissertation

August 2026

# Contents

# Chapter 1

# Introduction

An Asian option is a path-dependent derivative whose payoff depends on an average of the underlying asset price over a set of monitoring dates[cite: 1]. In the arithmetic case, the monitored prices are added and divided by the number of fixings before the strike is applied[cite: 1]. This averaging reduces the influence of an individual price observation, but it also makes the option value depend on how the asset prices at all the monitoring dates behave together, not just the price on a single date[cite: 1].

If the Asian option is modelled under geometric Brownian motion (GBM), the geometric average of the monitored prices is lognormally distributed and the corresponding option can be valued analytically[cite: 1]. The GBM framework is introduced in Section 2.3[cite: 1].

The arithmetic average is instead a sum of correlated lognormal variables and does not produce a convenient closed-form price[cite: 1]. Monte Carlo simulation is therefore a natural pricing method[cite: 1]. It can generate the complete asset-price path, evaluate the discounted payoff and approximate the required risk-neutral expectation[cite: 1]. This allows it to handle higher-dimensional pricing problems, such as Asian options with many monitoring dates, without first reducing the problem to a small number of variables (Kemna and Vorst, 1990; Glasserman, 2004)[cite: 1].

The main limitation of Monte Carlo simulation is its slow inverse square-root convergence rate[cite: 1]. This means that halving the sampling error requires approximately four times as many simulated paths[cite: 1]. This can become computationally expensive quickly[cite: 1]. That is why Variance-reduction techniques (VRTs) have been developed, they attempt to achieve the same or greater accuracy using fewer simulated paths at a lower computational cost[cite: 1]. This dissertation compares standard Monte Carlo with three VRTs: antithetic variates (AV), a geometric control variate (GCV) and a neural control variate (NCV)[cite: 1].

## 1.1 Motivation, Research Problem and Contribution

Variance-reduction techniques differ in both their statistical effectiveness and their computational requirements[cite: 1]. Antithetic variates and geometric control variates are relatively inexpensive to apply, whereas neural control variates require a neural network to be trained before pricing begins[cite: 1]. Previous research has shown that neural control variates can reduce sampling variance, but whether this improvement justifies their additional computational cost has received less attention[cite: 1].

El Filali Ech-Chafiq, Lelong and Reghai (2021) examine dimension-reduction and

direct neural controls across several payoffs and asset-pricing models[cite: 1]. This dissertation follows their direct neural-control approach, in which the expected output of the network can be calculated analytically[cite: 1]. Their efficiency analysis includes both training and pricing within an individual experiment, but the distinction between one-off training costs and the cost of repeated valuations is not its main focus[cite: 1]. Their parameter-robustness analysis also focuses mainly on dimension reduction rather than on reusing a previously trained network for new pricing problems[cite: 1].

This dissertation therefore examines whether the variance reduction achieved by neural control variates justifies their additional computational cost when compared with simpler variance-reduction techniques[cite: 1]. It also considers whether the initial training cost can be spread across repeated valuations by reusing a trained network for related option contracts[cite: 1].

## 1.2 Research question and objectives

The main research question is: Under what conditions can a trained neural control variate improve the statistical and computational efficiency of arithmetic Asian option pricing relative to standard Monte Carlo, antithetic variates and a geometric Asian control variate?[cite: 1]

Four objectives support this question[cite: 1]:

- Compare the variance, standard error and variance-reduction ratio produced by the four estimators under each monitoring profile[cite: 1].
- Distinguish equal-observation performance from efficiency under a common simulated-path budget[cite: 1].
- Compare one-time training and pilot costs with pricing-only marginal cost, and estimate the number of valuations required to recover setup cost[cite: 1].
- Examine whether a frozen reference network remains an effective control for related contracts without full retraining[cite: 1].

# Chapter 2

# The Pricing Problem

This chapter defines the option-pricing problem considered in this dissertation. It first explains the payoff of a discretely monitored arithmetic Asian call. It then introduces risk-neutral valuation and the geometric Brownian motion model used to simulate the underlying asset price. Finally, it explains why the arithmetic Asian option cannot generally be valued analytically under this model and discusses the limitations of the modelling assumptions.

## 2.1 Arithmetic Asian Options

A standard European call option has a payoff that depends only on the asset price at maturity. An Asian option is different because its payoff depends on the average asset price over a specified period. It is therefore a path-dependent option: the value depends on the prices observed throughout the monitoring period rather than on the final price alone.

Suppose the asset price is observed at equally spaced monitoring dates,

$$0 < t_1 < t_2 < \cdots < t_m = T.$$

The arithmetic average of the monitored prices is

$$\bar{S}_A = \frac{1}{m} \sum_{j=1}^{m} S_{t_j},$$

where $S_{t_j}$ is the asset price at monitoring date $t_j$. The payoff of a fixed-strike arithmetic Asian call at maturity is then

$$X_A = \max\left(\bar{S}_A - K, 0\right),$$

where $K$ is the strike price. If the arithmetic average is above the strike, the option pays the difference. If the average is below the strike, the option expires with no value.

The use of an average reduces the effect of any single price observation, but it also makes the pricing problem more difficult. Under geometric Brownian motion, each monitored asset price is lognormally distributed, but the arithmetic average is a sum of correlated lognormal variables. This average does not have a convenient known distribution, so the expected payoff cannot generally be calculated using a closed-form pricing formula (Kemna and Vorst 1990; Glasserman 2004). A numerical method is therefore required.

## 2.2 Risk Neutral Pricing

Risk-neutral valuation follows from the assumption that the market contains no arbitrage opportunities. An arbitrage is a trading strategy that requires no initial investment, cannot produce a loss and has some possibility of producing a profit. If two portfolios generate the same future cash flows, the absence of arbitrage therefore requires them to have the same value today.

Under suitable market assumptions, the absence of arbitrage allows asset prices to be represented under an equivalent risk-neutral probability measure, denoted by $\mathbb{Q}$. Under this measure, discounted asset prices are martingales. For a non-dividend-paying asset, the expected rate of return under $\mathbb{Q}$ is therefore the risk-free rate $r$, rather than the return that investors expect in the real world.

A derivative can then be valued by taking its expected payoff under $\mathbb{Q}$ and discounting this expectation at the risk-free rate. The time-zero value of the arithmetic Asian call is therefore

$$V_0 = e^{-rT}\mathbb{E}^{\mathbb{Q}}\left[X_A\right].$$

Using the payoff defined in Section 2.1 gives

$$V_0 = e^{-rT}\mathbb{E}^{\mathbb{Q}}\left[\max\left(\bar{S}_A - K, 0\right)\right],$$

where $T$ is the time to maturity and $\mathbb{E}^{\mathbb{Q}}$ denotes an expectation under the risk-neutral measure.

The use of $\mathbb{Q}$ does not assume that investors are genuinely indifferent to risk. Nor is it intended to predict how asset prices will behave in the real world. It is a mathematical measure used to obtain prices that are consistent with the absence of arbitrage. To evaluate the expectation above, a model is still required for the evolution of the asset price under $\mathbb{Q}$. In this dissertation, the asset price is modelled using geometric Brownian motion.

## 2.3 Geometric Brownian Motion

Geometric Brownian motion (GBM) is the asset-price model used throughout this dissertation. Under the risk-neutral measure, the asset price $S_t$ follows

$$dS_t = rS_t\,dt + \sigma S_t\,dW_t^{\mathbb{Q}},$$

where $r$ is the risk-free rate, $\sigma$ is the volatility and $W_t^{\mathbb{Q}}$ is a standard Brownian motion under the risk-neutral measure. The first term describes the expected growth of the asset price, while the second introduces random changes.

Both terms are proportional to the current price $S_t$. It is therefore more convenient to express the model in terms of proportional price changes:

$$\frac{dS_t}{S_t} = r\,dt + \sigma\,dW_t^{\mathbb{Q}}.$$

Applying Itô's lemma to $\ln S_t$ gives

$$d\ln S_t = \left(r - \frac{1}{2}\sigma^2\right)dt + \sigma\,dW_t^{\mathbb{Q}}.$$

Integrating between time 0 and time $t$ produces

$$S_t = S_0 \exp\left[\left(r - \frac{1}{2}\sigma^2\right)t + \sigma W_t^{\mathbb{Q}}\right].$$

The logarithm of $S_t$ is therefore normally distributed, meaning that $S_t$ itself is lognormally distributed. This ensures that the modelled asset price remains positive.

For simulation between consecutive equally spaced monitoring dates, let $t_0 = 0$ and define

$$\Delta t = t_j - t_{j-1} = \frac{T}{m}.$$

The solution can then be written as

$$S_{t_j} = S_{t_{j-1}} \exp\left[\left(r - \frac{1}{2}\sigma^2\right)\Delta t + \sigma\sqrt{\Delta t}\, Z_j\right],$$

where $Z_j$ is an independent standard normal random variable. Repeating this calculation across all monitoring dates generates one complete asset-price path. These simulated paths will later be used to calculate the arithmetic Asian option payoff.

GBM assumes that the risk-free rate and volatility remain constant and that log returns over non-overlapping time intervals are independent and normally distributed. These assumptions make the model analytically and computationally convenient, although they also limit its ability to reproduce all features of observed asset prices. The implications of these limitations are discussed in Section 2.5 (Black and Scholes 1973; Merton 1973; Glasserman 2004).

## 2.4 Arithmetic and Geometric Averaging under GBM

Under GBM, each monitored asset price $S_{t_j}$ is lognormally distributed. The prices at different monitoring dates are also correlated because they are generated by the same asset-price path. The arithmetic average,

$$\bar{S}_A = \frac{1}{m}\sum_{j=1}^{m} S_{t_j},$$

is therefore a sum of correlated lognormal random variables. A sum of lognormal variables is not generally lognormally distributed and does not have a convenient known distribution. As a result, the risk-neutral expectation

$$e^{-rT}\mathbb{E}^{\mathbb{Q}}\left[\max\left(\bar{S}_A - K, 0\right)\right]$$

cannot generally be evaluated using an exact closed-form formula. This is the reason a numerical pricing method is required for the arithmetic Asian option.

The geometric average of the same monitored prices is

$$\bar{S}_G = \left(\prod_{j=1}^{m} S_{t_j}\right)^{1/m}.$$

Taking logarithms converts the product into a sum:

$$\ln \bar{S}_G = \frac{1}{m}\sum_{j=1}^{m} \ln S_{t_j}.$$

Under GBM, the log prices are jointly normally distributed. A linear combination of jointly normal variables is also normally distributed, so $\ln \bar{S}_G$ is normal and $\bar{S}_G$ is lognormal. The corresponding geometric Asian call can therefore be valued analytically under GBM (Kemna and Vorst 1990).

This difference between the two averages is important for the later analysis. The arithmetic Asian option is the contract being priced, while the geometric Asian option provides an analytically tractable comparison based on the same asset-price path. Chapter 3 explains how this information can be used as a control variate when estimating the arithmetic Asian option price.

## 2.5 GBM Limitations

Modelling asset prices under GBM does not provide a complete description of asset-price behaviour. The model assumes that volatility and the risk-free rate remain constant, price paths are continuous, and log returns over non-overlapping periods are independent and normally distributed. In practice, asset returns can display time-varying volatility, heavier tails and sudden jumps. Commodity prices may also exhibit features such as mean reversion and seasonality that are not represented by the standard GBM model (Cont 2001; Schwartz 1997).

More complicated models can account for some of these features. For example, Heston (1993) allows volatility to change stochastically, while Merton (1976) introduces jumps into the asset-price process. These models are not considered here because the research question concerns the efficiency of variance-reduction methods under the standard GBM framework. Moving to another model would change both the simulated pricing problem and the availability of the analytical geometric control, and would therefore constitute a separate extension of the study.

The conclusions of this dissertation are consequently conditional on the risk-neutral GBM framework. Whether neural control variates remain computationally effective under more realistic asset-price models is left for future research.

# Chapter 3

# Monte Carlo and Classical Variance Reduction Methods

This chapter explains how Monte Carlo simulation is used to estimate the arithmetic Asian option price defined in Chapter 2. The chapter then introduces antithetic variates, the general control-variate method and the geometric Asian control variate. The neural control variate is considered separately in Chapter 4.

## 3.1 Standard Monte Carlo simulation

Monte Carlo simulation approximates the risk-neutral expectation by generating many possible asset-price paths and averaging the resulting discounted payoffs (Boyle 1977; Glasserman 2004).

For each simulated path $i$, independent standard normal variables are generated and used in the GBM equation from Section 2.3. The arithmetic average and discounted payoff for that path are then calculated as

$$Y_i = e^{-rT} \max\left(\frac{1}{m}\sum_{j=1}^{m} S_{t_j}^{(i)} - K, 0\right).$$

After simulating $N$ independent paths, the standard Monte Carlo estimator is

$$\widehat{V}_{\mathrm{MC}} = \frac{1}{N}\sum_{i=1}^{N} Y_i.$$

Each $Y_i$ is one possible discounted payoff under the risk-neutral measure. Their sample average provides an estimate of the option value $V_0$. Since the paths are independent and generated under the correct risk-neutral distribution, the estimator is unbiased:

$$\mathbb{E}^{\mathbb{Q}}\left[\widehat{V}_{\mathrm{MC}}\right] = V_0.$$

If the variance of an individual discounted payoff is $\sigma_Y^2$, the variance of the estimator is

$$\mathrm{Var}\left(\widehat{V}_{\mathrm{MC}}\right) = \frac{\sigma_Y^2}{N}.$$

The corresponding standard error is

$$\mathrm{SE}\left(\widehat{V}_{\mathrm{MC}}\right) = \frac{\sigma_Y}{\sqrt{N}}.$$

In practice, $\sigma_Y$ is unknown and is replaced by the sample standard deviation of the simulated payoffs. The central limit theorem then allows a confidence interval to be constructed around the estimated price.

The standard error decreases at the inverse square-root rate. Halving the standard error requires four times as many simulated paths, which increases the computational cost. Variance-reduction techniques aim to achieve the same accuracy using fewer paths.

## 3.2 Antithetic variates

Antithetic variates reduce variance by generating simulated paths in pairs. As explained in Section 2.3, each step of a simulated GBM path uses a random variable $Z_j$. This variable follows the standard normal distribution, which has a mean of zero and a variance of one. It represents the random price movement during that time interval: a positive value pushes the price upwards, while a negative value pushes it downwards.

The shocks used to generate one complete path can be collected into the vector

$$Z = (Z_1, \ldots, Z_m).$$

The antithetic method pairs this vector with

$$-Z = (-Z_1, \ldots, -Z_m).$$

Both vectors have the same probability distribution, but their shocks move in opposite directions.

Let $Y(Z)$ be the discounted option payoff generated using $Z$. One antithetic observation is

$$Y_{\mathrm{AV}}(Z) = \frac{Y(Z) + Y(-Z)}{2}.$$

After generating $N$ independent pairs, the antithetic estimator is

$$\widehat{V}_{\mathrm{AV}} = \frac{1}{N}\sum_{i=1}^{N} Y_{\mathrm{AV}}(Z_i).$$

Because $Z$ and $-Z$ have the same distribution, both payoffs have the same expected value. The antithetic estimator is therefore unbiased.

The method reduces variance when the two payoffs are negatively correlated. If one path produces a payoff above its expected value, the paired path will tend to produce a payoff below it. Averaging the two offsets part of their variation. If their correlation is denoted by $\rho$, then

$$\mathrm{Var}\left(Y_{\mathrm{AV}}\right) = \frac{\sigma_Y^2}{2}(1+\rho).$$

A more negative value of $\rho$ produces a larger variance reduction. At an equal budget of two path evaluations, antithetic variates are less efficient than two independent Monte Carlo paths if the paired payoffs are positively correlated.

Antithetic variates are simple to apply and require no training or analytically known option price (Hammersley and Morton 1956; Glasserman 2004). However, each antithetic observation requires two asset-price paths to be generated and evaluated. Comparisons with standard Monte Carlo must therefore account for the additional path.

## 3.3 Control variates

A control variate reduces the variance of a Monte Carlo estimate by removing variation captured by a related quantity whose expected value is already known. Each simulated payoff is paired with this related quantity so that some of the randomness shared by the two can be cancelled.

Let $Y$ denote the discounted arithmetic Asian option payoff and let $C$ denote a related random variable whose expected value,

$$\mu_C = \mathbb{E}[C],$$

is known. The control-variate observation is defined as

$$Y_{\mathrm{CV}} = Y - \beta(C - \mu_C),$$

where $\beta$ is the control coefficient. Since

$$\mathbb{E}[C - \mu_C] = 0,$$

the adjustment does not change the expected value:

$$\mathbb{E}\left[Y_{\mathrm{CV}}\right] = \mathbb{E}[Y].$$

The control variate therefore aims to reduce variance while leaving the option-price estimate unbiased.

The variance of the adjusted observation is

$$\mathrm{Var}\left(Y_{\mathrm{CV}}\right) = \mathrm{Var}(Y) + \beta^2 \,\mathrm{Var}(C) - 2\beta \,\mathrm{Cov}(Y, C).$$

Differentiating this expression with respect to $\beta$ and setting the result equal to zero gives the variance-minimising coefficient:

$$\beta^* = \frac{\mathrm{Cov}(Y, C)}{\mathrm{Var}(C)}.$$

Substituting $\beta^*$ back into the variance expression gives

$$\mathrm{Var}\left(Y_{\mathrm{CV}}\right) = \mathrm{Var}(Y)\left(1 - \rho_{Y,C}^2\right), \qquad \text{when } \beta = \beta^*,$$

where $\rho_{Y,C}$ is the correlation between $Y$ and $C$. A value of $|\rho_{Y,C}|$ close to one therefore produces a large variance reduction. The correlation may be positive or negative because the sign of $\beta^*$ adjusts accordingly.

In practice, the optimal coefficient is unknown and must be estimated from simulated data. It may be estimated using the same observations employed for pricing, although this can introduce a small finite-sample bias. Alternatively, it can be estimated from an independent pilot sample and then held fixed during final pricing (Glasserman 2004). This dissertation uses the second approach because it keeps coefficient estimation separate from final evaluation. The pilot sample introduces an additional simulation cost, which is included in the computational comparisons described in Chapter 5.

The main practical requirement is finding a control that is strongly correlated with the target payoff and has a known expectation. For arithmetic Asian options under GBM, the geometric Asian payoff provides a natural control. Its value is available analytically, and it is strongly related to the arithmetic payoff because both are calculated from the same asset-price path (Kemna and Vorst 1990). The geometric Asian control variate is introduced formally in the next section.

## 3.4 Geometric Asian control variate

The geometric Asian control variate uses the payoff of a geometric Asian option to reduce the variance of the arithmetic Asian option estimate. Both payoffs are calculated from the same simulated asset-price path, but the geometric Asian option has an analytically known value under GBM (Kemna and Vorst 1990).

For a path monitored at $m$ dates, the geometric average is

$$\bar{S}_G = \left( \prod_{j=1}^{m} S_{t_j} \right)^{1/m}.$$

The discounted geometric Asian call payoff is

$$C = e^{-rT} \max\left(\bar{S}_G - K, 0\right).$$

As shown in Section 2.4, $\ln \bar{S}_G$ is normally distributed under GBM. Its mean and variance are

$$\mu_G = \ln S_0 + \left(r - \frac{1}{2}\sigma^2\right) \frac{1}{m} \sum_{j=1}^{m} t_j$$

and

$$v_G = \frac{\sigma^2}{m^2} \sum_{j=1}^{m} \sum_{k=1}^{m} \min(t_j, t_k).$$

The analytical value of the geometric Asian call is therefore

$$V_G = e^{-rT} \left[ e^{\mu_G + \frac{1}{2} v_G} \Phi(d_1) - K \Phi(d_2) \right],$$

where

$$d_1 = \frac{\mu_G - \ln K + v_G}{\sqrt{v_G}}, \qquad d_2 = \frac{\mu_G - \ln K}{\sqrt{v_G}},$$

and $\Phi$ is the cumulative distribution function of the standard normal distribution.

Since $C$ is already discounted, its expected value is

$$\mathbb{E}[C] = V_G.$$

Let $Y$ denote the discounted arithmetic Asian payoff calculated from the same path. The geometric control-variate observation is

$$Y_{\mathrm{GCV}} = Y - \beta_G (C - V_G),$$

where $\beta_G$ is the control coefficient.

After estimating this coefficient from an independent pilot sample, the final estimator is

$$\widehat{V}_{\mathrm{GCV}} = \frac{1}{N} \sum_{i=1}^{N} \left[ Y_i - \widehat{\beta}_G (C_i - V_G) \right].$$

The arithmetic and geometric payoffs are strongly related because they are calculated using different averages of the same prices. The geometric payoff can therefore explain a large proportion of the variation in the arithmetic payoff. It is also inexpensive to

calculate because it uses the asset-price path that has already been simulated. Its main additional cost is the pilot sample used to estimate the control coefficient.

The geometric control variate provides a particularly strong classical benchmark in this dissertation. However, its analytical value depends on the structure of the Asian option and the GBM model. A similarly convenient control may not be available for a different payoff or asset-price model, which motivates the learned control introduced in Chapter 4.

# Chapter 4

# Neural Control Variates

The geometric control variate provides a strong and inexpensive benchmark for arithmetic Asian options under GBM. A neural control variate instead learns a control from simulated data but requires a neural network to be trained before pricing begins. This chapter explains how the neural control is constructed and why its expected output can be calculated analytically.

## 4.1 Learning a Control Variate

A neural network can be trained to approximate the relationship between the random inputs used to generate an asset-price path and the resulting discounted payoff. If the network reproduces changes in the payoff accurately, its output should be strongly correlated with the payoff and may therefore provide an effective control variate.

The neural network does not replace Monte Carlo simulation or directly determine the final option price. Instead, its output is used as the control $C$ in the control-variate estimator introduced in Section 3.3. Let $H_\theta(Z)$ denote the output of a neural network with fitted parameters $\theta$, where $Z$ is the vector of standard normal shocks used to generate the asset-price path. The corresponding neural control-variate observation is

$$Y_{\text{NCV}} = Y - \beta_H \left( H_\theta(Z) - \mathbb{E}[H_\theta(Z)] \right).$$

To use the network as a control variate, its expected output must be known. This is often difficult for a neural network because its output is a complicated function of its random inputs. The expectation could be estimated using a separate Monte Carlo simulation, but this would introduce another source of sampling error and additional computational cost. For the neural control to be useful in this study, its expectation must therefore be calculated accurately and efficiently. The chosen network architecture allows this expectation to be obtained analytically.

El Filali Ech-Chafiq, Lelong, and Reghai (2021) apply neural controls to option-pricing problems and propose a network whose expected output can be calculated analytically when the inputs are Gaussian. This dissertation follows their direct neural-control approach.

The network used here contains one hidden layer with a ReLU activation function and a linear output. This architecture is deliberately restricted so that its expected output remains analytically available. The network structure is introduced in the next section before the expectation is derived.

## 4.2 Network Architecture

The discounted arithmetic Asian payoff can be written as a function of the standard normal shocks used to generate the asset-price path:

$$Y = f(Z),$$

where

$$Z = (Z_1, \ldots, Z_m)^\top.$$

For each simulated path $i$, a new input vector

$$Z^{(i)} = \left(Z_1^{(i)}, \ldots, Z_m^{(i)}\right)^\top$$

is generated, containing one standard normal shock for each monitoring interval. This vector is used to generate one asset-price path and its discounted payoff,

$$Y^{(i)} = f\left(Z^{(i)}\right).$$

The training sample contains many of these input-payoff pairs. The purpose of the neural network is to learn the relationship between the simulated shocks $Z$ and the resulting discounted payoff $f(Z)$.

Following the direct neural-control construction of El Filali Ech-Chafiq, Lelong, and Reghai (2021), the network contains one hidden layer with $h$ neurons, a ReLU activation function and a linear output. It is written as

$$H_\theta(Z) = W_2 \,\mathrm{ReLU}(W_1 Z + b_1) + b_2,$$

where

$$W_1 \in \mathbb{R}^{h\times m}, \qquad b_1 \in \mathbb{R}^h, \qquad W_2 \in \mathbb{R}^{1\times h}, \qquad b_2 \in \mathbb{R}.$$

The collection of weights and biases is denoted by $\theta$.

The first layer forms a weighted combination of the inputs. Before applying the activation function, the value for hidden neuron $i$ is

$$A_i = w_i^\top Z + b_{1,i},$$

where $w_i^\top$ is row $i$ of $W_1$. The ReLU function is then applied:

$$\mathrm{ReLU}(A_i) = \max(A_i, 0).$$

Negative values are replaced by zero, while positive values remain unchanged. The linear output layer combines the activated neurons:

$$H_\theta(Z) = \sum_{i=1}^{h} W_{2,i} \max(A_i, 0) + b_2.$$

The network output is trained to follow changes in the discounted payoff $f(Z)$. It does not need to reproduce the payoff perfectly. For use as a control variate, it only needs to explain enough of the payoff variation to reduce the variance of the final estimator.

The shallow architecture is important because it allows the expected network output to be calculated from the fitted weights and biases. The next section derives this expectation.

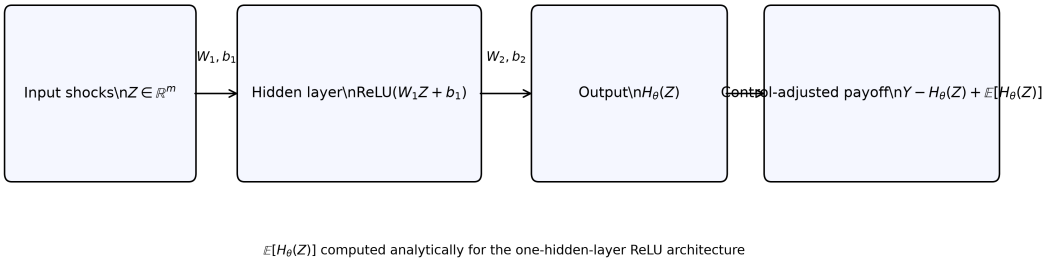


Figure 4.1: Architecture of the neural control variate.

## 4.3 Analytical Expectation of the Network Output

Once training is complete, the network weights and biases are fixed. The only remaining randomness comes from the input vector

$$Z \sim \mathcal{N}(0, I_m),$$

where $I_m$ is the $m \times m$ identity matrix. The elements of $Z$ are therefore independent standard normal random variables.

The network output can be written as

$$H_\theta(Z) = \sum_{i=1}^{h} W_{2,i} \max(A_i, 0) + b_2,$$

where

$$A_i = w_i^\top Z + b_{1,i}$$

is the value entering hidden neuron $i$ before ReLU is applied. Here, $w_i^\top$ is row $i$ of $W_1$.

The first step is to find the distribution of $A_i$. Since $A_i$ is a linear combination of normally distributed variables, it is also normally distributed:

$$A_i \sim \mathcal{N}(\mu_i, \sigma_i^2).$$

Its mean is

$$\begin{aligned} \mu_i &= \mathbb{E}[A_i] \\ &= \mathbb{E}\left[w_i^\top Z + b_{1,i}\right] \\ &= w_i^\top \mathbb{E}[Z] + b_{1,i}. \end{aligned}$$

Since $\mathbb{E}[Z] = 0$,

$$\mu_i = b_{1,i}.$$

Its variance is

$$\begin{aligned} \sigma_i^2 &= \mathrm{Var}(A_i) \\ &= \mathrm{Var}\left(w_i^\top Z + b_{1,i}\right) \\ &= \mathrm{Var}\left(w_i^\top Z\right). \end{aligned}$$

Adding the constant bias $b_{1,i}$ changes the mean but not the variance. Since the covariance matrix of $Z$ is $I_m$,

$$\begin{aligned}\sigma_i^2 &= w_i^\top I_m w_i \\ &= w_i^\top w_i \\ &= \sum_{j=1}^{m} W_{1,ij}^2.\end{aligned}$$

Here, $W_{1,ij}$ is the weight connecting input $Z_j$ to hidden neuron $i$.

The network applies ReLU to $A_i$:

$$\mathrm{ReLU}(A_i) = \max(A_i, 0).$$

The remaining problem is therefore to calculate the expected positive part of a normally distributed variable.

Suppose

$$X \sim \mathcal{N}(\mu, \sigma^2), \qquad \sigma > 0.$$

The standard normal density is

$$\phi(u) = \frac{1}{\sqrt{2\pi}} e^{-u^2/2},$$

and its cumulative distribution function is

$$\Phi(a) = \int_{-\infty}^{a} \phi(u)\, du.$$

Since $X^+ = \max(X, 0)$, negative values contribute nothing to its expectation. Therefore,

$$\mathbb{E}[X^+] = \int_0^\infty x f_X(x)\, dx,$$

where the density of $X$ is

$$f_X(x) = \frac{1}{\sigma}\phi\left(\frac{x-\mu}{\sigma}\right).$$

Substituting this density gives

$$\mathbb{E}[X^+] = \int_0^\infty x \frac{1}{\sigma}\phi\left(\frac{x-\mu}{\sigma}\right)\, dx.$$

Now make the substitution

$$u = \frac{x-\mu}{\sigma}.$$

Rearranging gives

$$x = \mu + \sigma u$$

and

$$dx = \sigma\, du.$$

When $x = 0$, the new lower limit is

$$u = -\frac{\mu}{\sigma}.$$

When $x$ approaches infinity, $u$ also approaches infinity. The expectation therefore becomes

$$\begin{aligned}\mathbb{E}[X^+] &= \int_{-\mu/\sigma}^{\infty} (\mu + \sigma u)\frac{1}{\sigma}\phi(u)\,\sigma\,du \\ &= \int_{-\mu/\sigma}^{\infty} (\mu + \sigma u)\phi(u)\,du.\end{aligned}$$

Separating the two terms gives

$$\mathbb{E}[X^+] = \mu\int_{-\mu/\sigma}^{\infty}\phi(u)\,du + \sigma\int_{-\mu/\sigma}^{\infty} u\phi(u)\,du.$$

Consider the first integral. From the definition of the standard normal CDF,

$$\int_{-\mu/\sigma}^{\infty}\phi(u)\,du = 1 - \Phi\left(-\frac{\mu}{\sigma}\right).$$

The standard normal distribution is symmetric, so

$$\Phi(-a) = 1 - \Phi(a).$$

It follows that

$$1 - \Phi\left(-\frac{\mu}{\sigma}\right) = \Phi\left(\frac{\mu}{\sigma}\right).$$

Therefore,

$$\int_{-\mu/\sigma}^{\infty}\phi(u)\,du = \Phi\left(\frac{\mu}{\sigma}\right).$$

For the second integral, differentiating the standard normal density gives

$$\phi'(u) = -u\phi(u).$$

Therefore,

$$u\phi(u) = -\phi'(u),$$

and hence

$$\begin{aligned}\int_{-\mu/\sigma}^{\infty} u\phi(u)\,du &= [-\phi(u)]_{-\mu/\sigma}^{\infty} \\ &= \phi\left(-\frac{\mu}{\sigma}\right).\end{aligned}$$

The standard normal density is also symmetric, meaning that

$$\phi(-a) = \phi(a).$$

Consequently,

$$\int_{-\mu/\sigma}^{\infty} u\phi(u)\,du = \phi\left(\frac{\mu}{\sigma}\right).$$

Substituting both results into the expectation gives

$$\mathbb{E}[X^+] = \sigma\phi\left(\frac{\mu}{\sigma}\right) + \mu\Phi\left(\frac{\mu}{\sigma}\right).$$

Applying this result to hidden neuron $i$ gives

$$\mathbb{E}\left[\mathrm{ReLU}(A_i)\right] = \sigma_i\phi\left(\frac{\mu_i}{\sigma_i}\right) + \mu_i\Phi\left(\frac{\mu_i}{\sigma_i}\right), \qquad \sigma_i > 0.$$

If $\sigma_i = 0$, then $A_i$ is deterministic and

$$\mathbb{E}\left[\mathrm{ReLU}(A_i)\right] = \max(\mu_i, 0).$$

The complete network output is a linear combination of these hidden-neuron outputs. Linearity of expectation therefore gives

$$\mathbb{E}\left[H_\theta(Z)\right] = b_2 + \sum_{i=1}^{h} W_{2,i}\left[\sigma_i \phi\left(\frac{\mu_i}{\sigma_i}\right) + \mu_i \Phi\left(\frac{\mu_i}{\sigma_i}\right)\right],$$

for hidden neurons with $\sigma_i > 0$, with the deterministic expression above used when $\sigma_i = 0$.

The hidden neurons do not need to be independent for this calculation. Linearity of expectation allows their expectations to be added even when their outputs are correlated.

The fitted network can now be centred around its analytically known expectation. With the control coefficient fixed at one, the neural control-variate observation is

$$Y_{\mathrm{NCV}} = f(Z) - H_\theta(Z) + \mathbb{E}[H_\theta(Z)].$$

Taking expectations gives

$$\begin{aligned}\mathbb{E}\left[Y_{\mathrm{NCV}}\right] &= \mathbb{E}[f(Z)] - \mathbb{E}[H_\theta(Z)] + \mathbb{E}[H_\theta(Z)] \\ &= \mathbb{E}[f(Z)].\end{aligned}$$

The adjustment therefore leaves the expected option price unchanged. If the network successfully follows variation in the payoff, subtracting its centred output can reduce the sampling variance. Crucially, its expectation is obtained directly from the fitted parameters.

## 4.4 Network Training

For each training path, the shock vector $Z^{(i)}$ is supplied to the network and the corresponding discounted payoff $f(Z^{(i)})$ is used as the target. The network produces its own output $H_\theta(Z^{(i)})$, and training aims to minimise the mean squared difference between the two:

$$\widehat{\theta} = \arg\min_{\theta} \frac{1}{N_{\mathrm{train}}} \sum_{i=1}^{N_{\mathrm{train}}} \left[f\left(Z^{(i)}\right) - H_\theta\left(Z^{(i)}\right)\right]^2.$$

The difference between the discounted payoff and the network output is the prediction error. During training, backpropagation calculates how each weight and bias affects the mean squared error. An optimisation algorithm then adjusts the parameters to reduce this error. The optimisation algorithm and training settings used in the experiments are reported in Section 5.2.

One complete pass through the training sample is called an epoch. A checkpoint is a saved version of the network weights and biases after a particular epoch. Saving checkpoints makes it possible to compare the network at different stages of training and select the version that performs best on unseen data.

The network does not need to reproduce every payoff exactly. It needs to capture enough of the variation in $f(Z)$ that the residual

$$f(Z) - H_\theta(Z)$$

has lower variance than the original payoff. Once the final checkpoint has been selected, its parameters are fixed and are not updated during pricing. The final pricing paths are generated independently of the training and validation samples.

Training creates a setup cost that is not present for standard Monte Carlo or antithetic variates. Whether the resulting variance reduction is large enough to justify this cost is examined in the experimental design and results chapters.

## 4.5 Reusing a Trained Network

Training a neural control variate creates a one-off computational cost. If the network is used to price only one option, this full cost must be justified by the variance reduction achieved for that valuation. If the same network can be reused, its training cost can instead be spread across several pricing problems.

Suppose a network $H_{\theta_0}$ has been trained for a reference contract. Reusing the network means applying it to a related target contract without changing its weights or biases. The target payoff changes, but the network remains fixed or frozen.

Let $f_q(Z)$ denote the discounted payoff of target contract $q$. The frozen network can first be applied with a control coefficient fixed at one:

$$Y_{\text{frozen},1} = f_q(Z) - H_{\theta_0}(Z) + \mathbb{E}\left[H_{\theta_0}(Z)\right].$$

This tests whether the network output remains useful for the target contract without any additional fitting. Its expectation remains available analytically because the network parameters have not changed.

A change in the contract may alter the scale of the relationship between the network output and the target payoff. A target-specific coefficient can therefore be estimated. The population variance-minimising coefficient is

$$\beta_q^* = \frac{\text{Cov}\left(f_q(Z), H_{\theta_0}(Z)\right)}{\text{Var}\left(H_{\theta_0}(Z)\right)}.$$

In practice, this coefficient is estimated from an independent target-specific pilot sample and then held fixed during final pricing. The resulting observation is

$$Y_{\text{frozen},\beta} = f_q(Z) - \widehat{\beta}_q\left[H_{\theta_0}(Z) - \mathbb{E}\left[H_{\theta_0}(Z)\right]\right].$$

Estimating $\widehat{\beta}_q$ can rescale the frozen network when a useful relationship remains. However, it cannot recover payoff information that the reference network did not learn. It also requires a target-specific pilot sample, which creates an additional cost.

Direct reuse requires the reference and target contracts to have the same number of monitoring dates. This is because a network trained with $m$ monitoring dates expects an input vector containing $m$ shocks. The strike, volatility and maturity may change while the number of monitoring dates remains fixed. However, a network trained for a 12-date contract cannot be applied directly to a 252-date contract because the input vector has a different length. The specific contracts used in the reuse analysis are described in Chapter 5.

# Chapter 5

# Experimental Design

This chapter explains how the four pricing methods are compared. It describes the contracts being priced, the neural-network configuration, the allocation of simulated paths and the measures used to assess statistical and computational efficiency.

## 5.1 Contract specifications and monitoring profiles

The reference contract has an initial asset price of $S_0 = 100$, a strike of $K = 100$, a risk-free rate of $r = 0.05$, a volatility of $\sigma = 0.20$ and a maturity of $T = 1$ year.

Six additional contracts are created by changing the strike, volatility or maturity one at a time. All other parameters remain equal to those of the reference contract.

| Contract | $K$ | $\sigma$ | $T$ |
|---|---|---|---|
| Reference | 100 | 0.20 | 1.0 |
| Low strike | 80 | 0.20 | 1.0 |
| High strike | 120 | 0.20 | 1.0 |
| Low volatility | 100 | 0.10 | 1.0 |
| High volatility | 100 | 0.50 | 1.0 |
| Short maturity | 100 | 0.20 | 0.5 |
| Long maturity | 100 | 0.20 | 2.0 |

This gives a seven-contract grid in which the effect of changing each contract parameter can be examined separately. The initial asset price and risk-free rate remain fixed at $S_0 = 100$ and $r = 0.05$ throughout.

Each contract is evaluated using 12 and 252 equally spaced monitoring dates. For the one-year reference contract, these correspond approximately to monthly and business-daily monitoring. When maturity changes, the number of dates remains fixed but the time between them changes.

The two monitoring profiles are analysed separately. A 12-date network receives 12 shocks, while a 252-date network receives 252 shocks. Networks are therefore trained and reused within each profile, rather than transferred between them.

## 5.2 Selection of the neural-network configurations

The neural network used for the 12-date profile has 12 inputs and 449 trainable parameters. By comparison, using the same hidden width of 32 for the 252-date profile produces

8,129 parameters. Training both networks with 5,000 simulated paths therefore provides approximately 11.14 paths per parameter for the 12-date network but only 0.62 for the 252-date network.

The initial 252-date network performed poorly when pricing compared to the GCV. To examine whether this was caused by the size of the network relative to the training sample, an analysis of different neural-network configurations was conducted. Hidden-layer widths of 8, 16 and 32 were tested using 5,000, 10,000 and 20,000 training paths.

For each configuration, checkpoints were saved at several stages during training. A checkpoint is a saved copy of the network weights and biases after a given number of epochs. Each checkpoint was evaluated using 10,000 independently simulated validation paths. The checkpoint producing the lowest average variance of

$$f(Z) - H_\theta(Z)$$

was selected, where $f(Z)$ is the discounted option payoff and $H_\theta(Z)$ is the network output.

Table X reports the variance-reduction ratio relative to standard Monte Carlo. A larger ratio indicates greater variance reduction.

| Hidden width | 5,000 training paths | 10,000 training paths | 20,000 training paths |
|---|---|---|---|
| 8 | 309 | 969 | 2,393 |
| 16 | 273 | 1,004 | 3,306 |
| 32 | 227 | 609 | 3,510 |

Increasing the amount of training data improved every network. With 5,000 paths, performance became worse as the hidden width increased. The width-8 network achieved a variance-reduction ratio of 309, compared with 227 for the width-32 network. All three networks also selected the relatively early checkpoint at epoch 25.

At 20,000 training paths, widths 16 and 32 performed similarly. Width 32 produced the higher average variance-reduction ratio, at 3,510 compared with 3,306. However, width 16 produced a slightly lower mean residual variance, at 0.02006 compared with 0.02086. The confidence intervals for residual variance also overlapped considerably:

$$[0.01695,\ 0.02316]$$

for width 16 and

$$[0.01379,\ 0.02792]$$

for width 32.

Width 16 was selected for the final 252-date experiments. It achieved similar variance reduction to width 32 while using 4,065 rather than 8,129 trainable parameters. This configuration is treated as the best configuration selected from the sensitivity analysis, rather than as a globally optimal neural network.

The final network configurations are summarised below.

| Monitoring dates | Hidden width | Parameters | Training paths | Paths per parameter | Selected checkpoint |
|---|---|---|---|---|---|
| 12 | 32 | 449 | 5,000 | 11.14 | 1,000 |
| 252 | 16 | 4,065 | 20,000 | 4.92 | 200 |

These configurations are used in the subsequent comparisons with standard Monte Carlo, antithetic variates and the geometric control variate.

## 5.3 Pricing comparison

The final neural-network configurations were compared with standard Monte Carlo, antithetic variates and the geometric control variate. Each method was applied to the seven contracts described in Section 5.1 under both monitoring profiles. The complete experiment was repeated ten times using different simulated samples.

Two comparisons were conducted. The first gave each method 50,000 final pricing observations. This allows their statistical performance to be compared using the same number of observations, but it does not account for the additional paths required to set up each method.

Standard Monte Carlo required no setup sample and used 50,000 independently simulated paths. The geometric control variate used 1,000 pilot paths to estimate its control coefficient, followed by 50,000 pricing paths. The neural control variate required either 5,000 or 20,000 training paths, depending on the monitoring profile, followed by 50,000 pricing paths.

For antithetic variates, one observation is the average payoff from a path generated using $Z$ and its paired path generated using $-Z$. Consequently, 50,000 antithetic observations require 100,000 simulated paths.

| Method | Setup paths | Pricing observations | Total path evaluations |
|---|---|---|---|
| Monte Carlo | 0 | 50,000 | 50,000 |
| Antithetic variates | 0 | 50,000 pairs | 100,000 |
| Geometric control variate | 1,000 | 50,000 | 51,000 |
| Neural control variate, 12 dates | 5,000 | 50,000 | 55,000 |
| Neural control variate, 252 dates | 20,000 | 50,000 | 70,000 |

A second comparison used a fixed total budget of 50,000 simulated paths. Setup paths were included within this budget. Monte Carlo therefore retained all 50,000 paths for pricing, while antithetic variates used 25,000 pairs. After allowing for setup, the geometric control variate used 49,000 pricing paths. The 12-date neural control variate used 45,000 pricing paths, while the 252-date neural control variate used 30,000.

| Method | Setup paths | Pricing observations | Total path budget |
|---|---|---|---|
| Monte Carlo | 0 | 50,000 | 50,000 |
| Antithetic variates | 0 | 25,000 pairs | 50,000 |
| Geometric control variate | 1,000 | 49,000 | 50,000 |
| Neural control variate, 12 dates | 5,000 | 45,000 | 50,000 |
| Neural control variate, 252 dates | 20,000 | 30,000 | 50,000 |

The equal-observation comparison measures how much variance each method removes once it is available for pricing. The equal-budget comparison also accounts for the simulated paths used to estimate the geometric control coefficient or train the neural network.

## 5.4 Measuring statistical and computational performance

The methods are first compared using their standard errors. For an estimator with observation variance $s^2$ and $N$ pricing observations, the estimated variance of the option-

price estimate is

$$\frac{s^2}{N},$$

and its standard error is

$$\mathrm{SE} = \frac{s}{\sqrt{N}}.$$

A lower standard error indicates a more precise option-price estimate.

Variance reduction is reported relative to standard Monte Carlo using the variance-reduction ratio

$$\mathrm{VRR} = \frac{s^2_{\mathrm{MC}}}{s^2_{\mathrm{method}}}.$$

A ratio greater than one means that the method produces a lower observation variance than standard Monte Carlo. For example, a variance-reduction ratio of 100 means that the observation variance is 100 times smaller.

Computational performance is assessed using measured runtime. Setup time and pricing time are recorded separately. Setup includes neural-network training or estimation of the geometric control coefficient, while pricing time includes only the generation and evaluation of the final pricing observations. This distinction allows the one-off cost of setting up a method to be separated from the cost of using it for subsequent valuations.

## 5.5 Statistical results

Variance-reduction ratios are summarised using geometric means across the independent replications. The NCV/GCV column compares the two control variates directly. A value greater than one indicates that NCV has lower variance than GCV.

### 5.5.1 The 12-date profile

Table X reports the results for the 12-date network, which was trained using 5,000 paths for 1,000 epochs.

| Contract | AV/MC | GCV/MC | NCV/MC | NCV/GCV |
|---|---|---|---|---|
| Reference | 4.16 | 1,290.2 | 22,810.8 | 17.68 |
| Low strike | 56.70 | 1,629.3 | 25,913.5 | 15.91 |
| High strike | 2.12 | 232.7 | 3,131.8 | 13.46 |
| Low volatility | 6.43 | 4,588.5 | 53,795.8 | 11.72 |
| High volatility | 3.00 | 201.4 | 3,565.3 | 17.70 |
| Short maturity | 4.05 | 2,698.6 | 30,167.7 | 11.18 |
| Long maturity | 4.29 | 598.4 | 13,949.1 | 23.31 |

All three methods reduced variance relative to standard Monte Carlo. Antithetic variates produced the smallest improvement for most contracts, while GCV produced much larger reductions. However, NCV substantially outperformed both classical methods across the complete contract grid.

Relative to GCV, the variance reduction achieved by NCV ranged from 11.18 for the short-maturity contract to 23.31 for the long-maturity contract. NCV produced lower variance than GCV in every replication for every contract, and all confidence intervals for the NCV/GCV ratio remained above one.

For the reference contract, the standard error fell from 0.038122 under standard Monte Carlo to 0.018688 using antithetic variates and 0.001061 using GCV. NCV reduced it further to 0.000255. The 12-date results therefore provide strong evidence that a contract-specific neural control can outperform the geometric Asian control when the network is trained successfully.

### 5.5.2 The 252-date profile

Table X reports the results for the final 252-date network. This network had a hidden width of 16 and was trained using 20,000 paths for 200 epochs. All ten replications were completed successfully.

| Contract | AV/MC | GCV/MC | NCV/MC | NCV/GCV | 95% CI |
|---|---|---|---|---|---|
| Reference | 4.16 | 1,307.7 | 2,970.5 | 2.27 | [1.99, 2.60] |
| Low strike | 65.51 | 1,509.6 | 1,752.1 | 1.16 | [0.92, 1.47] |
| High strike | 2.11 | 201.5 | 216.8 | 1.08 | [0.91, 1.27] |
| Low volatility | 6.40 | 4,687.4 | 4,942.9 | 1.05 | [0.84, 1.32] |
| High volatility | 3.01 | 204.7 | 589.5 | 2.88 | [2.45, 3.38] |
| Short maturity | 4.04 | 2,739.5 | 4,141.1 | 1.51 | [1.32, 1.73] |
| Long maturity | 4.29 | 605.3 | 2,015.2 | 3.33 | [2.87, 3.86] |

NCV clearly outperformed GCV for the reference, high-volatility, short-maturity and long-maturity contracts. Its largest relative advantage occurred for the long-maturity contract, where its observation variance was approximately 3.33 times lower than that of GCV.

The average NCV/GCV ratios also exceeded one for the two strike changes and the low-volatility contract. However, their confidence intervals include one, so these results do not establish a clear difference between the methods. There was no contract for which the results provided clear evidence that GCV had lower variance than NCV.

Across both monitoring profiles, the geometric control remained a strong classical benchmark. The point estimates favoured NCV for all seven 252-date contracts, although clear evidence of an advantage was obtained for only four contracts. Whether this statistical improvement justifies the additional training cost is considered in the next section.

## 5.6 Computational performance

The variance-reduction results do not account for the additional cost of setting up each method. GCV requires a small pilot sample to estimate its control coefficient, while NCV requires the neural network to be trained before pricing begins. This training cost is paid once, after which the fitted network can be used for repeated valuations of the same contract.

Table X summarises the measured runtimes. Standalone runtime includes both setup and pricing. Marginal pricing time measures the cost of an additional valuation after setup has been completed.

For the 12-date profile, training the neural network required approximately 29.5–29.9 seconds. A standalone NCV valuation was therefore considerably slower than GCV, which required less than 0.06 seconds. NCV was also slightly slower when both methods used

| Monitoring dates | GCV standalone | NCV standalone | GCV marginal pricing | NCV marginal pricing | Matched-accuracy break-even |
|---|---|---|---|---|---|
| 12 | 0.048–0.056 s | 29.5–29.9 s | 0.046–0.052 s | 0.055–0.058 s | 609–703 valuations |
| 252 | 1.1–1.4 s | 37–41 s | 1.12–1.34 s | 0.80–0.98 s | 40–101 valuations |

the same number of pricing observations. Its marginal pricing time was approximately 0.055–0.058 seconds, compared with 0.046–0.052 seconds for GCV.

The computational advantage of the 12-date NCV came from its substantially lower variance. It could achieve the same standard error as GCV using fewer pricing paths. Under this matched-accuracy comparison, the initial training cost was recovered after 609–703 repeated valuations, depending on the contract.

| Contract | Matched-accuracy break-even |
|---|---|
| Reference | 688 |
| Low strike | 684 |
| High strike | 676 |
| Low volatility | 703 |
| High volatility | 626 |
| Short maturity | 694 |
| Long maturity | 609 |

At 1,000 repeated valuations, the projected runtime saving from using NCV rather than GCV was approximately 26%–37%.

The 252-date profile produced a different runtime pattern. Training the network required approximately 36–40 seconds, meaning that GCV remained much faster for a single valuation. Once training was complete, however, NCV pricing required approximately 0.80–0.98 seconds, compared with 1.12–1.34 seconds for GCV.

The 252-date NCV therefore had both a lower marginal pricing time and lower observed variance. When both methods used the same number of pricing observations, the projected break-even points across the alternative contracts ranged from 84 to 167 repeated valuations. When the methods were compared at the accuracy achieved by GCV, the range fell to 40–101 valuations.

These results show that NCV was not computationally attractive for a one-off valuation. Its training cost could only be justified when the fitted network was used repeatedly, with the required number of valuations depending on its variance reduction and marginal pricing cost.

The reported runtimes are specific to the implementation and hardware used in this study. Neural-network speed can be affected by a range of variables including the software library, batching procedure, degree of vectorisation, numerical precision and the use of CPU or GPU hardware. A more highly optimised implementation could therefore produce different training times and break-even points. However, all methods were evaluated using the same codebase, hardware and timing procedure, providing a consistent comparison within the experimental setting.

The practical relevance of these break-even points depends on how often the trained network remains suitable for pricing. If changes in market or contract parameters require a new network, the same pricing problem may not be repeated often enough to recover the training cost. This motivates the network-reuse experiment considered in the next section.

## 5.7 Reusing the neural network

To test whether initial training cost could be shared, the network trained for the reference contract was reused to price the six alternative contracts. Its weights and biases were held fixed, meaning that no further neural-network training was performed.

Two approaches were tested. The first reused the network with the control coefficient fixed at $\beta = 1$. The second estimated a new value of $\beta$ using a 1,000-path pilot sample from the target contract. This allowed the output of the reference network to be rescaled without retraining it.

The reused network was compared with a target-specific GCV. The comparison is reported as

$$R_{\text{reuse/GCV}} = \frac{\text{Var(GCV)}}{\text{Var(reused NCV)}}.$$

A value greater than one means that the reused neural control produced lower variance than GCV. A value below one means that GCV produced lower variance.

| Target contract | $\beta = 1$, 12 dates | Estimated $\beta$, 12 dates | $\beta = 1$, 252 dates | Estimated $\beta$, 252 dates |
|---|---|---|---|---|
| Low strike | 0.0031 | 0.0040 | 0.0033 | 0.0042 |
| High strike | 0.0008 | 0.0086 | 0.0007 | 0.0092 |
| Low volatility | 0.0003 | 0.0106 | 0.0003 | 0.0107 |
| High volatility | 0.0123 | 0.1267 | 0.0120 | 0.1247 |
| Short maturity | 0.0016 | 0.8410 | 0.0016 | 0.6436 |
| Long maturity | 0.0155 | 1.8304 | 0.0153 | 1.3693 |

Reusing the network with $\beta = 1$ performed poorly under both monitoring profiles. None of the fixed-coefficient controls outperformed GCV. Under the 252-date profile, fixed-coefficient reuse also performed worse than standard Monte Carlo for the high-strike contract.

Estimating a target-specific value of $\beta$ substantially improved the reused control. Relative to fixing the coefficient at one, it reduced residual variance by a geometric-mean factor of approximately 26.6 for the 12-date profile and 24.9 for the 252-date profile. Estimated-$\beta$ reuse also reduced variance relative to standard Monte Carlo for every target contract.

The estimated coefficients varied considerably across the contract grid. Under the 252-date profile, their mean values ranged from 0.207 for the high-strike contract to 2.594 for the high-volatility contract. Similar variation occurred under the 12-date profile. This explains why a coefficient fixed at one did not transfer successfully when the contract parameters changed.

Despite the improvement from estimating $\beta$, GCV remained the stronger control for five of the six target contracts under both monitoring profiles. The long-maturity contract was the only target for which reuse outperformed GCV under both monitoring profiles. Its variance advantage over GCV was 1.83 under the 12-date profile and 1.37 under the 252-date profile.

This result was consistent across all replications for the 12-date profile and all replications for the 252-date profile. Both maturity changes also transferred more successfully than the strike and volatility changes, although only two maturity changes were tested, so no general conclusion about maturity transfer can be drawn.

The results therefore show that estimating $\beta$ can correct differences in the scale of the network output, but it cannot fully adapt a network trained for one contract to a different payoff.

Direct reuse of a fixed network was not a reliable replacement for contract-specific training or GCV across the tested contract grid. However, the improvement from estimating $\beta$, together with the successful long-maturity transfer, indicates that the trained network retained some information that could be reused. Alternative approaches to adapting the network are considered in the discussion and suggestions for future research.

# Chapter 6

# Limitations and Conclusion

## 6.1 Limitations

The results are conditional on the risk-neutral GBM model. GBM provides a controlled setting in which paths can be simulated exactly and the geometric Asian option price is available analytically. However, the conclusions may change under models containing stochastic volatility, jumps or other features not represented by GBM.

The experiments considered only arithmetic Asian call options and a grid of seven contracts. Strike, volatility and maturity were changed separately, so the study did not examine contracts in which several parameters changed simultaneously. The effectiveness of the methods should therefore not be assumed to extend to every option-pricing problem.

The network sensitivity analysis was also limited to one shallow ReLU architecture, three hidden widths and three training-sample sizes. The selected networks were the best configurations tested rather than globally optimal networks. Since NCV performance was sensitive to the amount of training data and the size of the network, other architectures or training procedures could produce different results.

The 12-date and 252-date results should not be interpreted as showing that monitoring frequency alone caused the difference in performance. Changing the number of monitoring dates also changed the payoff, the number of network inputs and the selected network configuration.

The break-even calculations assume that the trained network remains suitable for repeated valuations. In practice, changes in spot price, volatility, interest rates or time to maturity may create a different pricing problem. The reuse experiment showed that a fixed network generally did not transfer successfully using only a new control coefficient. The reported break-even points therefore demonstrate the potential benefit of repeated use, but they do not establish that these savings would always be achieved in practice.

Finally, GCV provides an unusually strong benchmark for arithmetic Asian options under GBM. This makes the comparison demanding, but also specific to a pricing problem for which a highly effective analytical control is available. The relative value of NCV may be different for payoffs or pricing models where no comparable classical control exists.

## 6.2 Future research

The most important extension would be to develop a neural control that can be used across a wider range of contracts. Rather than training the network using only the simulated shocks, strike, volatility, maturity, interest rate and spot price could also be included as inputs. The network could then be trained across a range of contract and market conditions instead of being fitted to one fixed pricing problem.

This would allow a larger network to be trained in advance, possibly during periods when computational demand is lower, and then used for valuations as market conditions change. A small target-specific pilot sample could still be used to estimate the control coefficient. Provided that the same shallow ReLU architecture is retained and the contract parameters are fixed during each valuation, the network's expected output would remain analytically available. Further work would be required to determine whether this remains practical for more flexible architectures.

A second approach would be to partially retrain an existing network for a new contract. This may require fewer simulated paths and less time than training a new network from the beginning. The performance and cost of partial retraining could then be compared with direct reuse, contract-specific training and GCV.

The analysis could also be extended beyond GBM and arithmetic Asian calls. Testing the method under stochastic-volatility or jump models would show whether the results remain valid in a more complicated pricing setting. Other path-dependent derivatives could be considered, particularly where no analytical control as effective as the geometric Asian option is available.

Finally, a wider network-selection study could examine additional architectures, training-sample sizes, regularisation methods and optimisation procedures. This would help determine whether the configurations used in this dissertation can be improved and how much training data are required as the dimension of the pricing problem increases.

## 6.3 Conclusion

This dissertation examined whether neural control variates can improve the statistical and computational efficiency of Monte Carlo pricing for arithmetic Asian options. Standard Monte Carlo, antithetic variates, the geometric control variate and a neural control variate were compared using both 12-date and 252-date monitoring profiles.

The contract-specific NCV produced the greatest average variance reduction across the tested contracts. Its advantage over GCV was particularly strong for the 12-date profile. For the 252-date profile, it clearly outperformed GCV for four contracts. For the remaining three contracts, the point estimates favoured NCV, but the results did not establish a clear difference between the methods. The sensitivity analysis also showed that this performance depended on selecting a suitable network size and providing sufficient training data.

The computational results were more conditional. GCV was the more efficient method for a one-off valuation because it required very little setup. Once the neural network had been trained, however, NCV could provide a more efficient valuation at a matched level of accuracy. Its initial training cost could therefore be justified when the same network was used sufficiently many times.

The reuse experiments showed that a network trained for one contract did not generally remain competitive with GCV when transferred to other contracts using only a new

control coefficient. The long-maturity contract was the exception, with reuse outperforming GCV under both monitoring profiles. The practical benefit of NCV is therefore currently greatest when the same pricing problem is repeated, rather than when contract or market parameters change frequently.

Overall, neural control variates are not a replacement for simpler variance-reduction techniques for pricing arithmetic Asian options under GBM. They can provide substantial statistical and computational benefits when an accurately trained network is already available and can be used repeatedly. For isolated valuations, or where frequent changes require the network to be retrained, GCV remains the more practical method.

# References


Black, Fischer and Myron Scholes (1973). "The Pricing of Options and Corporate Liabilities". In: *Journal of Political Economy* 81.3, pp. 637–654. DOI: 10.1086/260062.

Boyle, P. P. (1977). "Options: A Monte Carlo Approach". In: *Journal of Financial Economics* 4.3, pp. 323–338. DOI: 10.1016/0304-405X(77)90005-8.

Cont, Rama (2001). "Empirical Properties of Asset Returns: Stylized Facts and Statistical Issues". In: *Quantitative Finance* 1.2, pp. 223–236. DOI: 10.1080/713665670.

El Filali Ech-Chafiq, Zakaria, Jérôme Lelong, and Aryane Reghai (2021). "Automatic Control Variates for Option Pricing Using Neural Networks". In: *Monte Carlo Methods and Applications* 27.2, pp. 91–104. DOI: 10.1515/mcma-2020-2081.

Glasserman, Paul (2004). *Monte Carlo Methods in Financial Engineering*. Springer. DOI: 10.1007/978-0-387-21617-1.

Hammersley, J. M. and K. W. Morton (1956). "A New Monte Carlo Technique: Antithetic Variates". In: *Mathematical Proceedings of the Cambridge Philosophical Society* 52.3, pp. 449–475. DOI: 10.1017/S0305004100031455.

Heston, Steven L. (1993). "A Closed-Form Solution for Options with Stochastic Volatility with Applications to Bond and Currency Options". In: *The Review of Financial Studies* 6.2, pp. 327–343. DOI: 10.1093/rfs/6.2.327.

Kemna, A. G. Z. and A. C. F. Vorst (1990). "A Pricing Method for Options Based on Average Asset Values". In: *Journal of Banking & Finance* 14.1, pp. 113–129. DOI: 10.1016/0378-4266(90)90039-5.

Merton, Robert C. (1973). "Theory of Rational Option Pricing". In: *The Bell Journal of Economics and Management Science* 4.1, pp. 141–183. DOI: 10.2307/3003143.

— (1976). "Option Pricing When Underlying Stock Returns Are Discontinuous". In: *Journal of Financial Economics* 3.1–2, pp. 125–144. DOI: 10.1016/0304-405X(76)90022-2.

Schwartz, Eduardo S. (1997). "The Stochastic Behavior of Commodity Prices: Implications for Valuation and Hedging". In: *The Journal of Finance* 52.3, pp. 923–973. DOI: 10.1111/j.1540-6261.1997.tb02721.x.